

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра САПР

ОТЧЁТ
по учебной практике (технологической (проектно-технологической)
практике)

Тема: Использование средств визуализации и объектно-ориентированного программирования для проектирования программных систем на языке C#

Студент гр. 4352

Даричев Е.М.

Преподаватель

Калмычков В.А.

Санкт-Петербург

2026

ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ (ТЕХНОЛОГИЧЕСКУЮ (ПРОЕКТНО-ТЕХНОЛОГИЧЕСКУЮ) ПРАКТИКУ)

Студент Даричев Е.М.

Группа 4352

Тема практики: Использование средств визуализации и объектно-ориентированного программирования для проектирования программных систем на языке C#

Задание на практику:

1. Ознакомление с основными возможностями визуализации в среде C#. В рамках первого задания необходимо изучить примеры использования графических компонентов Windows Forms: работу с кнопками, элементами управления, изображениями, меню, а также базовые графические примитивы (линии, фигуры, текст). Данное задание закладывает фундамент для последующих практических работ.
2. Разработка программы анимации движения объектов по математической траектории. Во втором задании требуется реализовать приложение, в котором геометрические объекты перемещаются по кривой, заданной уравнением $x^3 + y^3 = 1$ (вариант 27). Объекты включают окружность и шестиугольник с независимыми параметрами движения, пульсации и вращения. Программа должна поддерживать интерактивное управление параметрами через панель управления и обеспечивать гладкую анимацию.
3. Разработка программы для построения и визуализации фракталов. В третьем задании необходимо реализовать систему рекурсивного построения фракталов с визуализацией в двух представлениях: основное окно для отображения фрактала с интерактивным управлением и дополнительное окно для представления иерархической структуры рекурсивного дерева. Тип фрактала: снежинка из окружностей.

Сроки прохождения практики: 13.07.2026

Дата сдачи отчёта: 13.07.2026

Дата защиты отчёта: 13.07.2026

Студент гр. 4352

Даричев Е.М.

Преподаватель

Калмычков В.А.

АННОТАЦИЯ

В данной работе рассматривается использование средств визуализации и объектно-ориентированного программирования для проектирования программных систем на языке C#.

Первое задание посвящено ознакомлению с графическими компонентами Windows Forms: элементами управления, работой с изображениями, обработкой событий мыши и базовыми графическими примитивами (линии, окружности, многоугольники, текст).

Второе задание — разработка программы анимации, в которой геометрические объекты движутся по кривой $x^3 + y^3 = 1$. Программа использует равномерное распределение точек по длине дуги, демонстрирует применение синусоидальных функций и геометрических преобразований. Интерфейс включает панель управления параметрами в режиме реального времени и интерактивное управление через мышь.

Третье задание — разработка программы для построения фракталов с рекурсивной структурой узлов. Программа позволяет управлять глубиной рекурсии, масштабным коэффициентом и параметрами визуализации. Особенностью является двойная система координат и два окна: главное для управления фракталом и дополнительное для визуализации иерархической структуры дерева.

SUMMARY

This work explores the use of visualization tools and object-oriented programming for designing software systems in C#.

The first task focuses on familiarization with Windows Forms graphical components: UI controls, image processing, mouse event handling, and fundamental graphical primitives (lines, circles, polygons, text).

The second task involves developing an animation program where geometric objects move along the curve $x^3 + y^3 = 1$. The program implements uniform arc-length distribution for smooth movement, demonstrates sinusoidal functions and geometric transformations. The interface includes a control panel for real-time parameter adjustment and interactive mouse-based control.

The third task focuses on developing a fractal construction program with recursive node structures. The program allows managing recursion depth, scale factors, and visualization parameters. A distinctive feature is the dual coordinate system with two windows: the main window for fractal manipulation and an auxiliary window for displaying the hierarchical tree structure.

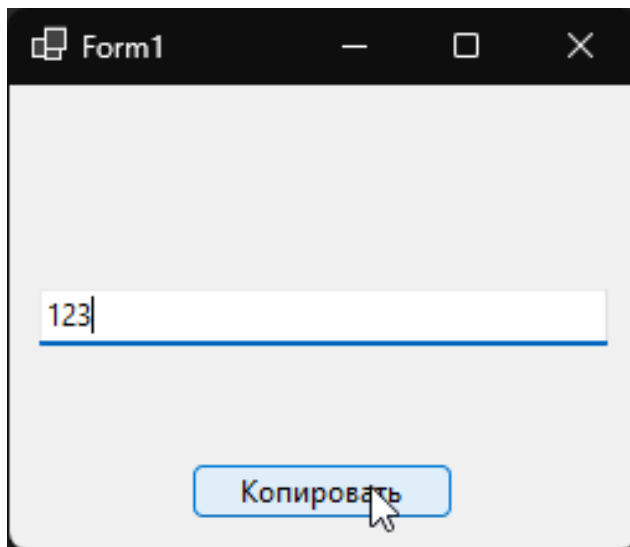
Содержание

АННОТАЦИЯ	3
Введение	5
Раздел 1. Знакомство с ознакомительными материалами	5
Задание 1.1 Простые Кнопки	5
Задание 1.2 Сообщение	5
Задание 1.3 Работа с изображениями	6
Задание 1.4 Файлы и меню	7
Задание 1.5 Работа с графикой	8
Задание 1.6 Движение мыши	9
Задание 1.7 Простая траектория	9
Раздел 2. Первое индивидуальное Задание	10
1. Формулировка задания	10
2. Математическая постановка	10
3. Описание используемых примитивов	15
4. Код программы	16
5. Примеры различного использования	28
Раздел 3. Второе индивидуальное Задание	29
1. Формулировка задания	29
2. Математическая постановка	29
3. Описание используемых элементов для интерфейса и их настройка	31
4. Описание используемых примитивов	32
5. Код программы	33
6. Примеры различного использования	40
Раздел 4. Заключение	41

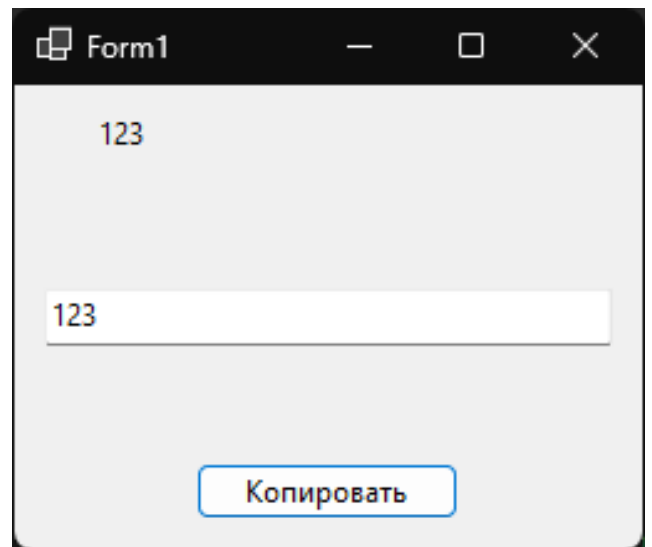
Введение

Раздел 1. Знакомство с ознакомительными материалами

Задание 1.1 Простые Кнопки

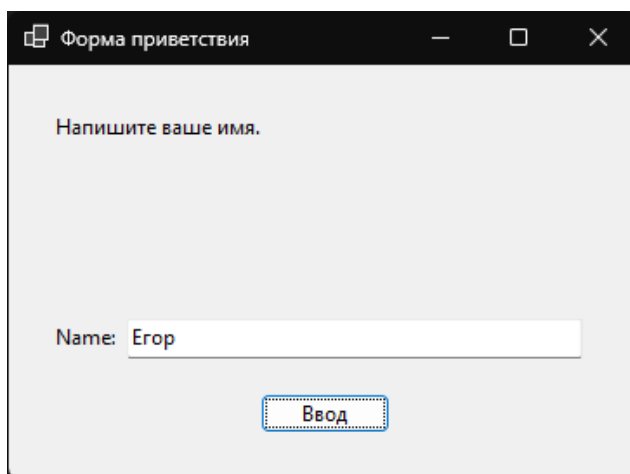


До

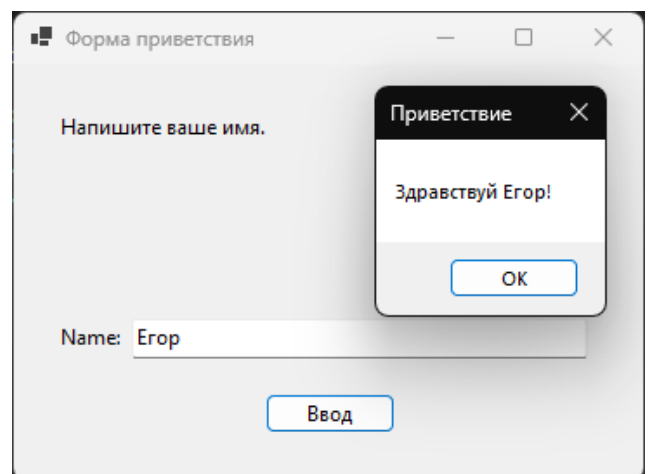


После

Задание 1.2 Сообщение



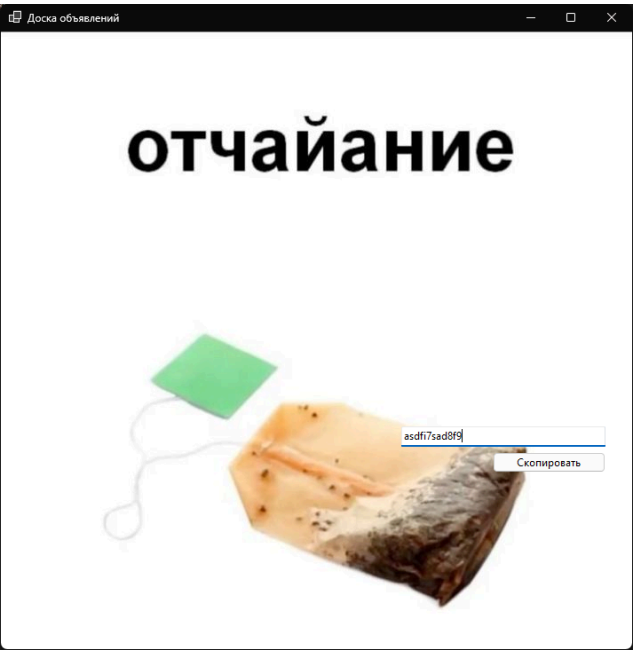
До



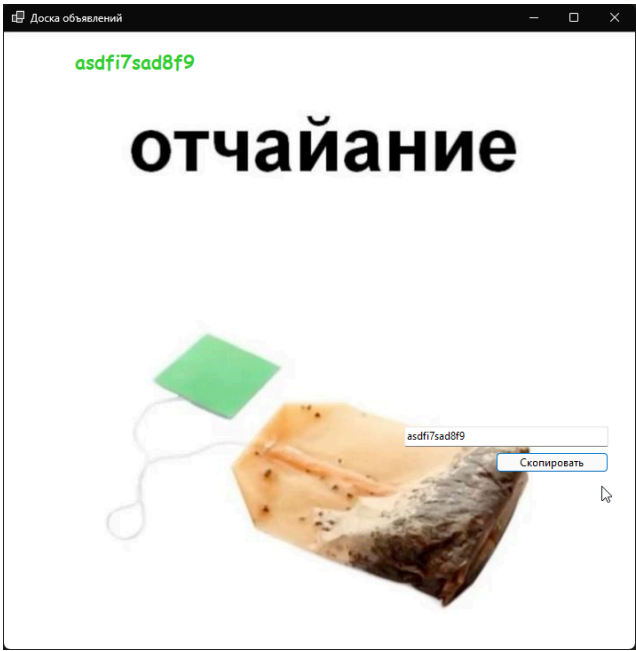
После

Задание 1.3 Работа с изображениями

Фоновое изображение

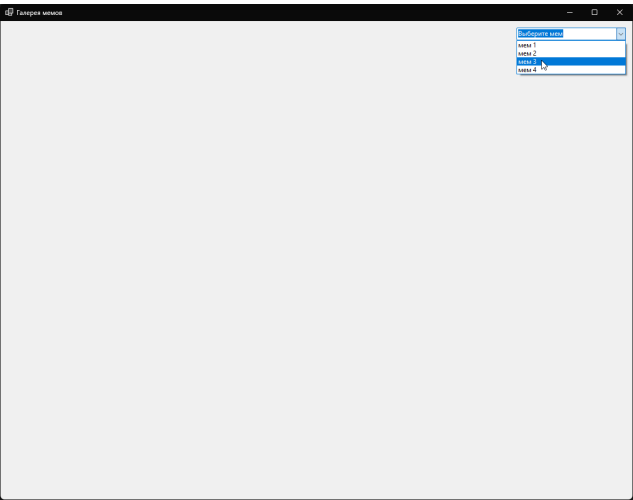


До

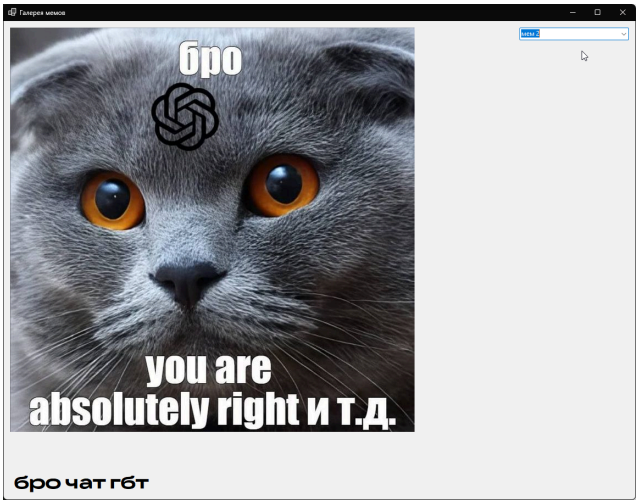


После

Выбор изображений из КомбоБокса

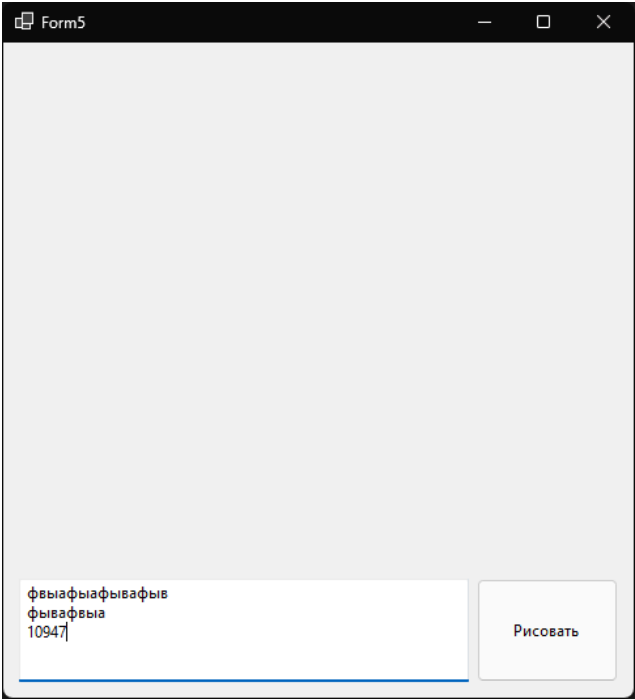


До

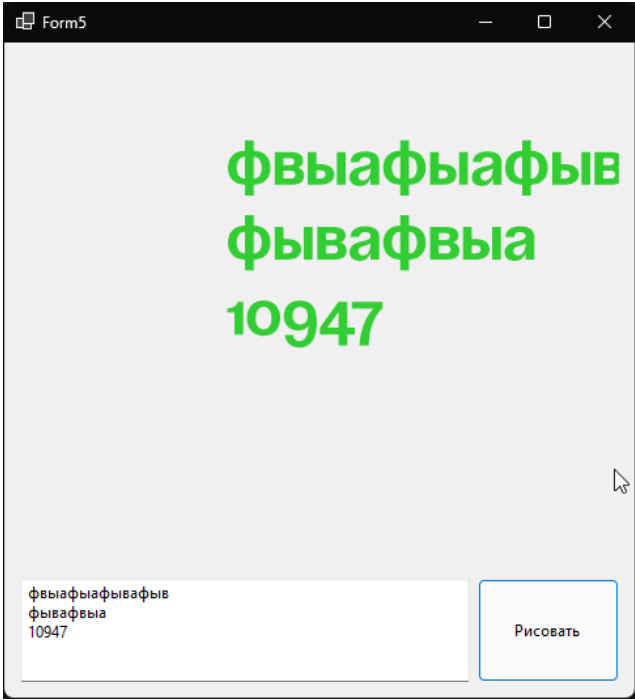


После

Рисование текста

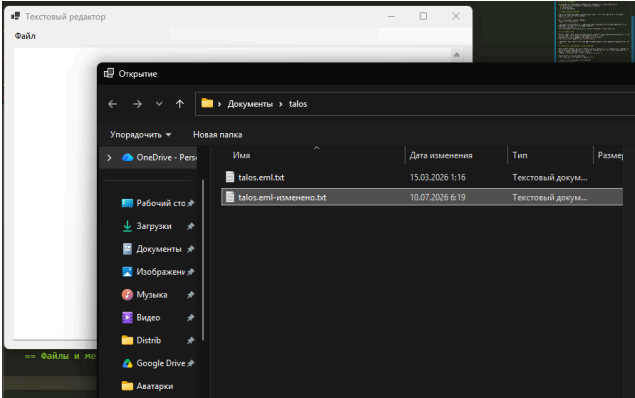


До

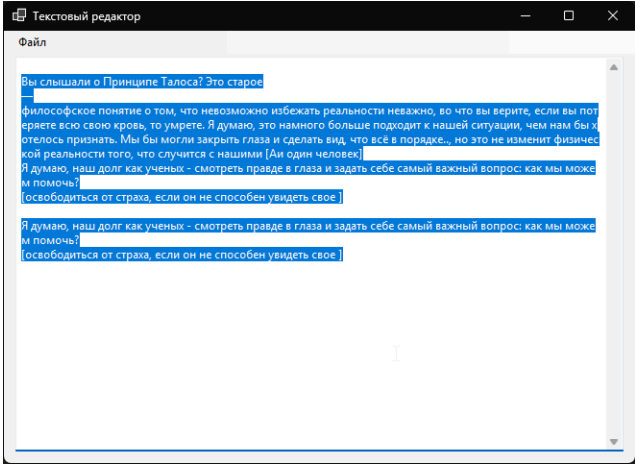


После

Задание 1.4 Файлы и меню



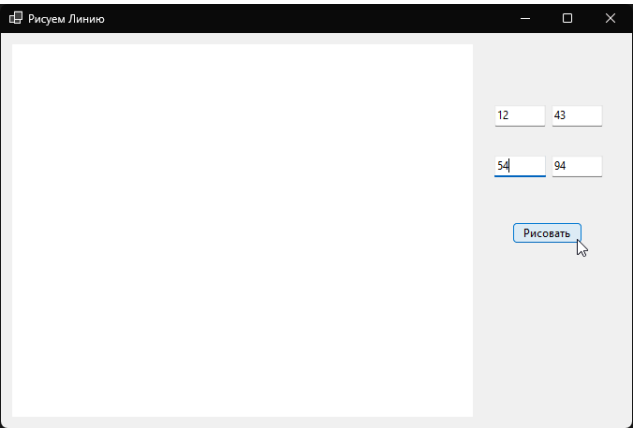
До



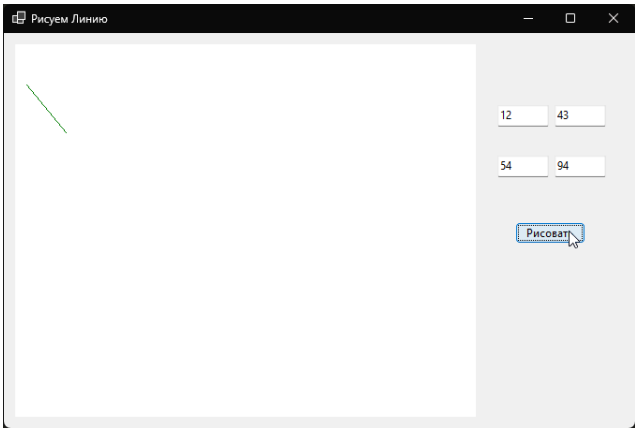
После

Задание 1.5 Работа с графикой

Рисование линии

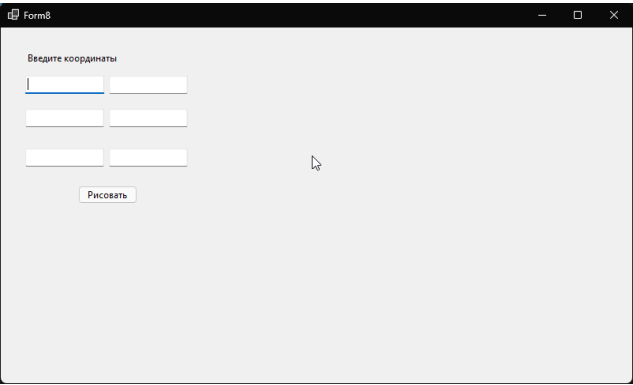


До

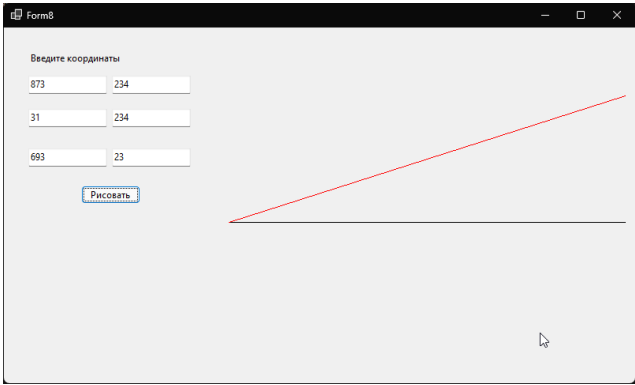


После

Треугольник

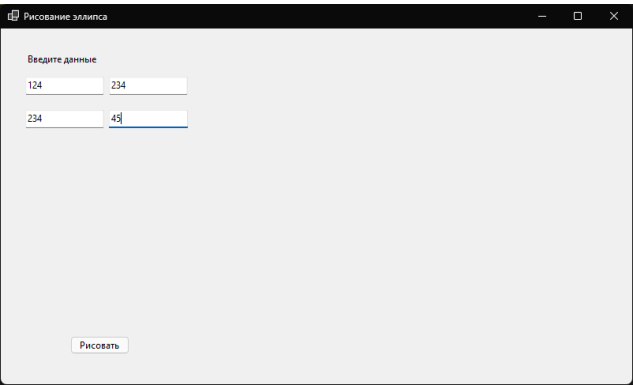


До

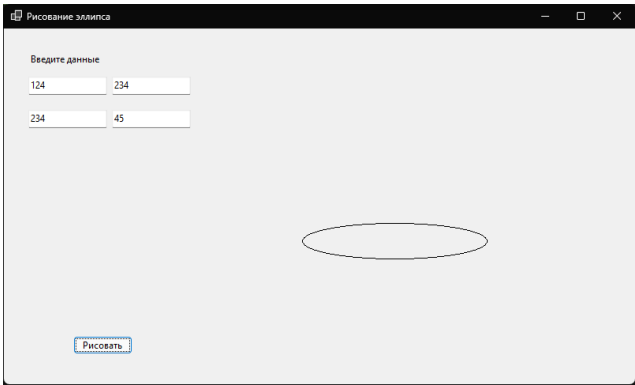


После

Эллипс

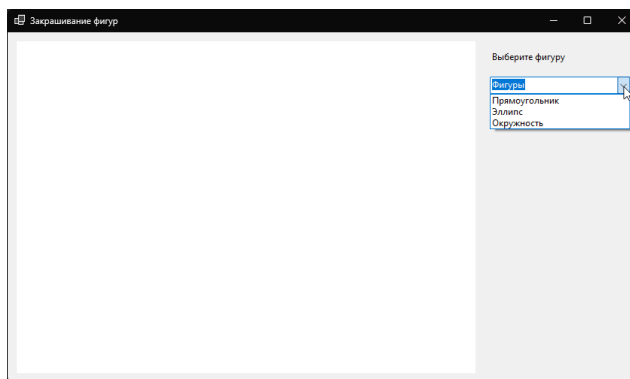


До

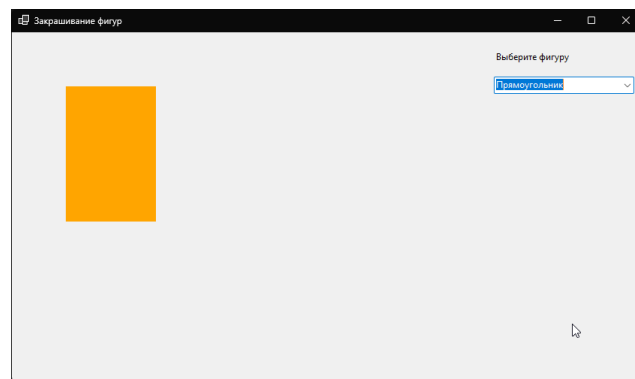


После

Заливка



До

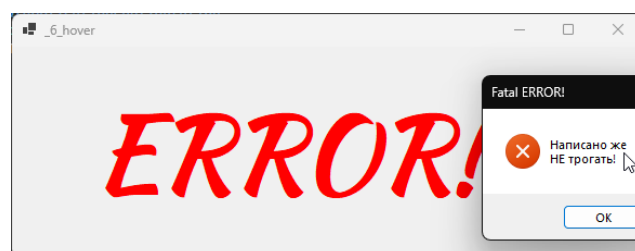


После

Задание 1.6 Движение мыши

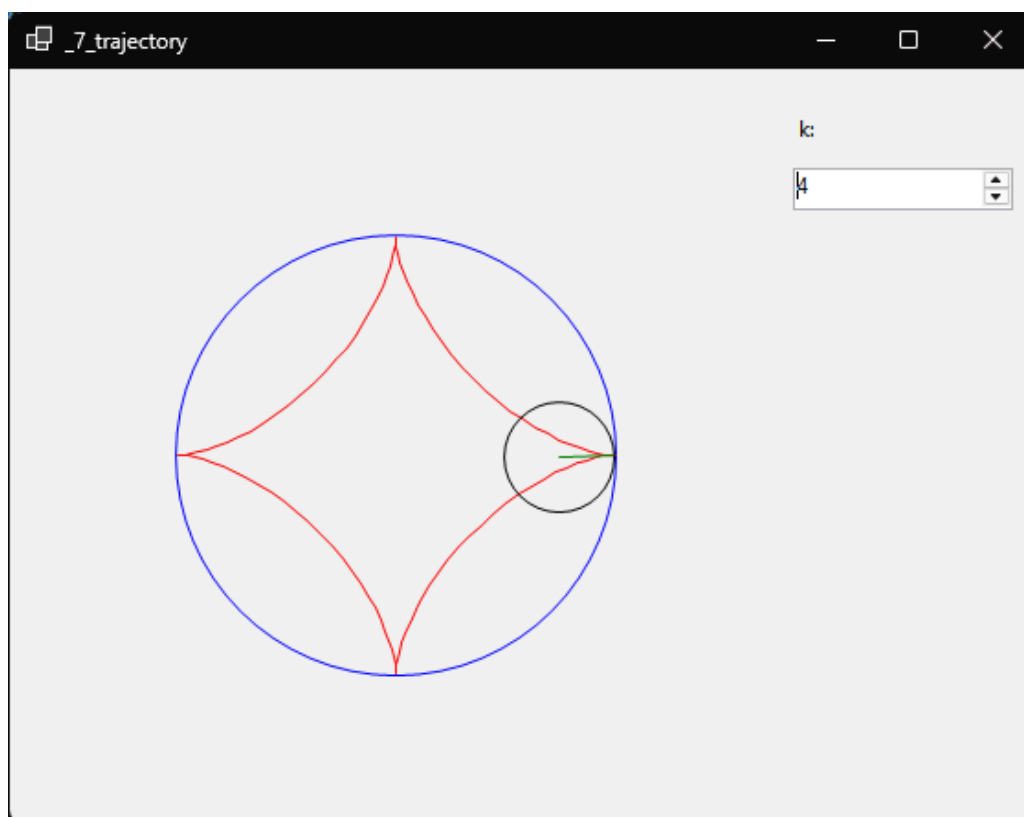


До



После

Задание 1.7 Простая траектория



Раздел 2. Первое индивидуальное Задание

1. Формулировка задания

В рамках первого индивидуального задания были реализованы три графических примера:

- 27. По траектории движется $x^3 + y^3 = 1$
- 11. Шестиугольник с различными параметрами фигуры с видом
- 3. Контур без заливки

2. Математическая постановка

В программе Task2 используется математическая модель, в которой кривая, по которой движутся объекты, строится на основе уравнения

$$x^3 + y^3 = 1.$$

Из этого уравнения выражается функция

$$y(x) = (1 - x^3)^{\frac{1}{3}}.$$

Поскольку в коде учитывается и отрицательная область значений, для вычисления кубического корня используется выражение с учетом знака аргумента:

$$y = (1 - x^3)^{\frac{1}{3}},$$

которое применяется в программе для корректного вычисления точки при $x^3 > 1$.

Построение кривой

Для построения траектории по интервалу $[x_{\min}, x_{\max}]$ программа выбирает последовательность точек

$$x_i = x_{\min} + \frac{i}{N}(x_{\max} - x_{\min}), \quad i = 0, 1, \dots, N,$$

где N — число шагов разбиения. Для каждой точки вычисляется

$$P_i = (x_i, y_i), \quad y_i = (1 - x_i^3)^{\frac{1}{3}}.$$

Полученная кривая является дискретной аппроксимацией гладкой кривой, заданной уравнением $x^3 + y^3 = 1$.

Равномерное распределение точек по длине дуги

Чтобы объекты двигались по кривой равномерно, а не с разной скоростью в зависимости от шага по оси x , программа не использует просто

равномерную сетку по x , а строит точки по длине дуги. Для каждого соседнего участка вычисляется длина

$$\Delta s_i = \sqrt{(x_{i+1} - x_i)^2 + (y_{i+1} - y_i)^2}.$$

Далее находится суммарная длина дуги:

$$S_i = \sum_{j=0}^{i-1} \Delta s_j, \quad S_0 = 0.$$

После этого точки распределяются по шагу h :

$$s_{k+1} = s_k + h.$$

Если точка лежит между P_i и P_{i+1} , то используется линейная интерполяция:

$$P(s) = P_i + \lambda(P_{i+1} - P_i),$$

$$\lambda = \frac{s - S_i}{S_{i+1} - S_i}.$$

Таким образом достигается равномерное перемещение по траектории независимо от её локального изгиба.

Движение окружности по траектории

Окружность перемещается по точкам траектории с индексом

$$\text{index_circle} = ((\lfloor T_c \rfloor \bmod N + N) \bmod N),$$

где T_c — текущий параметр времени объекта, увеличивающийся на величину скорости движения.

Положение окружности определяется как

$$P_t^o = P_{\text{index_circle}}.$$

То есть на каждом шаге анимации выбирается одна из точек траектории, соответствующая текущему моменту времени.

Изменение радиуса пульсации окружности

Радиус окружности меняется по закону синусоиды:

$$r(t) = r_{\min} + \frac{r_{\max} - r_{\min}}{2} (1 + \sin(\omega_p t)).$$

Здесь:

- $r_{\min}$ — минимальный радиус,

- $r_{\max}$ — максимальный радиус,
- ω_p — частота пульсации,
- t — глобальное время анимации.

Из этой формулы следует, что радиус колеблется между значениями

$$r_{\min} \leq r(t) \leq r_{\max}.$$

При $\sin(\omega_p t) = 1$ радиус достигает максимума, а при $\sin(\omega_p t) = -1$ — минимума.

Движение шестиугольника по траектории

Шестиугольник также перемещается по той же траектории, но с собственным параметром времени:

$$\text{index_hex} = ((\lfloor T_h \rfloor \bmod N + N) \bmod N).$$

Его центр помещается в точку касания

$$P_{\text{hex}(t)} = P_{\text{index_hex}}.$$

Пульсация и вращение шестиугольника

Размер шестиугольника меняется по закону

$$r_{\text{hex}(t)} = R_h - R_h |\sin(\omega_h t)|,$$

где R_h — исходный радиус описанной окружности, а ω_h — частота пульсации.

Из этой формулы видно, что в моменты времени, когда $\sin(\omega_h t) = 0$, радиус становится равным R_h , а при $\sin(\omega_h t) = 1$ он уменьшается до нуля.

Поворот шестиугольника определяется углом

$$\varphi(t) = \omega_\varphi t,$$

где ω_φ — угловая скорость вращения.

Координаты вершин шестиугольника вычисляются как

$$V_{k(t)} = \left(r_{\text{hex}(t)} \cos\left(\varphi(t) + k\frac{\pi}{3}\right), r_{\text{hex}(t)} \sin\left(\varphi(t) + k\frac{\pi}{3}\right) \right), \quad k = 0, \dots, 5.$$

Затем каждая вершина сдвигается в точку касания:

$$V_k^{\text{screen}(t)} = V_{k(t)} + P_{\text{hex}(t)}.$$

Преобразование математических координат в экранные

Для отображения на форме используется преобразование из математической системы координат в экранную:

$$x_{\text{screen}} = \frac{W}{2} + (x_{\text{math}} - x_c)z,$$

$$y_{\text{screen}} = \frac{H}{2} - (y_{\text{math}} - y_c)z,$$

где:

- W, H — размеры области рисования,
- (x_c, y_c) — центр обзора,
- z — коэффициент масштабирования.

Таким образом, вся анимация строится на сочетании геометрических преобразований, тригонометрических функций и дискретного движения по заранее вычисленной траектории.

Описание используемых элементов для интерфейса и их настройка (свойства)

Интерфейс программы Task2 реализован в виде Windows Forms-окна с левой панелью управления и правой областью рисования. Основная структура интерфейса задаётся контейнером `TableLayoutPanel`, который делит форму на две части: узкую панель настроек и большую область отображения.

Основная область рисования

В правой части формы расположен элемент `PictureBox`. Он используется как область вывода анимации. Для него заданы следующие параметры:

- свойство `Dock = Fill`, благодаря которому элемент занимает всё доступное место в правой колонке;
- обработчик события `Paint`, в котором выполняется отрисовка осей, сетки, траектории и движущихся объектов;
- поддержка мыши: обработчики `MouseDown`, `MouseMove`, `MouseUp` и `MouseWheel` позволяют перемещать область просмотра и изменять масштаб.

Панель управления

Слева размещена панель `panelControls`, представляющая собой вертикальный контейнер `FlowLayoutPanel`. Она содержит все элементы управления и разбита на логические блоки:

- блок управления симуляцией;
- блок параметров окружности;

- блок параметров шестиугольника;
- блок параметров траектории.

В верхней части панели находятся кнопки Start/Pause и Reset. Кнопка Start/Pause запускает или приостанавливает анимацию, а кнопка Reset возвращает параметры времени к начальному состоянию.

Блок управления симуляцией

Этот блок содержит:

- кнопку bStartPause для запуска/паузы анимации;
- кнопку bReset для сброса анимации;
- текстовое поле tObjectSpeed, задающее скорость перемещения объектов по траектории.

Свойства элементов настроены так, чтобы они были компактными и удобно располагались в одной строке или блоке. Для кнопок используются стандартные визуальные стили Windows Forms.

Блок параметров окружности

В этом блоке находятся элементы управления для первого объекта — круга:

- текстовые поля tCircleSmallR и tCircleBigR — минимальный и максимальный радиусы пульсации;
- текстовое поле tCirclePulseSpeed — скорость изменения радиуса;
- ComboBox cbCircleBrush — выбор кисти для контура окружности;
- метки для обозначения назначения каждого поля.

Эти элементы позволяют изменять параметры движения и визуального отображения окружности без изменения исходного кода.

Блок параметров шестиугольника

Для второго объекта — шестиугольника — предусмотрены следующие элементы:

- текстовое поле tHexagonR — радиус описанной окружности;
- текстовое поле tHexagonAngleSpeed — скорость вращения;
- текстовое поле tHexSpeed — скорость пульсации;
- ComboBox cbHexagonBrush — выбор кисти для контура шестиугольника.

С помощью этих полей можно управлять как геометрией, так и динамикой фигуры во время анимации.

Блок параметров траектории

Блок траектории отвечает за построение и отображение кривой. Здесь расположены:

- NumericUpDown nXMin и nXMax — границы интервала по оси x ;
- NumericUpDown nSteps — количество точек для разбиения кривой;
- NumericUpDown numericUpDown1 — шаг по длине дуги;
- NumericUpDown nCenterX и nCenterY — центр математической системы координат;
- NumericUpDown nZoom — масштаб отображения;
- ComboBox cbCurveBrush — выбор кисти для линии траектории.

Данные элементы изменяют параметры генерации траектории и её отображения на экране. При изменении любого из этих значений вызывается пересчет траектории и перерисовка изображения.

Меню кистей

В верхней части окна расположено меню «Кисти». Оно содержит команду «Создать кисть...». При выборе этой команды открывается диалоговое окно BrushDialog, где пользователь может задать:

- цвет контура;
- толщину линии;
- стиль линии (сплошная, пунктирная, точечная, штрихпунктирная, пользовательский);
- пользовательский шаблон штриховки.

Созданные кисти добавляются в список доступных вариантов для круга, шестиугольника и траектории.

Отображение формулировки

По нажатию на пункт меню открывается диалоговое окно с изображением задания и формулировкой.

Взаимодействие с мышью и масштабированием

Интерфейс поддерживает интерактивное управление просмотром:

- при зажатии левой кнопки мыши и перемещении курсора по PictureBox происходит сдвиг центра координат;
- колесо мыши изменяет масштаб отображения.

Это позволяет пользователю приближать и отдалять изображение, а также перемещать его в нужную область без изменения самой математической модели.

3. Описание используемых примитивов

В программе использовались базовые графические примитивы, которые рисуются через методы класса Graphics. Основными из них являются:

- линии: метод DrawLine используется для построения осей координат и линий сетки. С его помощью на фоне формы рисуются вертикальные и горизонтальные разметки, а также координатные оси;
- ломаная кривая: метод DrawLines применяется для отображения траектории, заданной последовательностью точек. В программе массив точек преобразуется из математической системы координат в экранные и соединяется непрерывной линией;
- окружность: метод DrawEllipse используется для рисования контура движущегося круга, а FillEllipse — для закраски его маркера в точке положения объекта;
- многоугольник: метод DrawPolygon применяется для построения шестиугольника, вершины которого вычисляются по формулам вращения и масштабирования;
- точечный маркер: метод FillEllipse используется для отображения контрольной точки на траектории и центра движущихся фигур.

Таким образом, в программе реализованы именно те примитивы, которые необходимы для анимации: линия, ломаная, окружность, многоугольник и точечные маркеры. Важной особенностью является то, что все примитивы строятся не вручную, а по заранее рассчитанным координатам, что обеспечивает плавное движение и корректное преобразование в экранную систему координат.

4. Код программы

MainForm

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Drawing.Drawing2D;
using System.Text;
using System.Windows.Forms;

namespace c2_SP_Tasks.Task2
{
    public partial class MainForm : Form
    {
        TrajectoryPath trajectory = new TrajectoryPath();

        PointF viewCenter = new PointF(-0.5f, 0.5f);
        float zoom = 140.0f;

        float globalTime = 0.0f;
        double circleT = 0;
        double hexagonT = 200;

        List<CustomBrushData> customBrushes = new List<CustomBrushData>();
        CustomBrushData defaultCircleBrush;
        CustomBrushData defaultHexagonBrush;
```



```

CustomBrushData defaultCurveBrush;

private Point mouseDragStart;
private PointF viewCenterDragStart;
private bool isDragging = false;

public MainForm()
{
    InitializeComponent();
}

private void ConfigureInputControls()
{
    nXMin.Minimum = -100;
    nXMin.Maximum = 100;
    nXMin.DecimalPlaces = 2;
    nXMin.Increment = 0.5m;
    nXMin.Value = -3.5m;

    nXMax.Minimum = -100;
    nXMax.Maximum = 100;
    nXMax.DecimalPlaces = 2;
    nXMax.Increment = 0.5m;
    nXMax.Value = 2.5m;

    nSteps.Minimum = 10;
    nSteps.Maximum = 100000;
    nSteps.Increment = 100m;
    nSteps.Value = 4000m;

    numericUpDown1.Minimum = 0.001m;
    numericUpDown1.Maximum = 2.0m;
    numericUpDown1.DecimalPlaces = 3;
    numericUpDown1.Increment = 0.005m;
    numericUpDown1.Value = 0.010m;

    nCenterX.Minimum = -1000;
    nCenterX.Maximum = 1000;
    nCenterX.DecimalPlaces = 3;
    nCenterX.Increment = 0.1m;
    nCenterX.Value = -0.5m;

    nCenterY.Minimum = -1000;
    nCenterY.Maximum = 1000;
    nCenterY.DecimalPlaces = 3;
    nCenterY.Increment = 0.1m;
    nCenterY.Value = 0.5m;

    nZoom.Minimum = 5;
    nZoom.Maximum = 50000;
    nZoom.Increment = 10m;
    nZoom.Value = 140m;
}

private void BindControlsEvents()
{
    nXMin.ValueChanged += (s, e) => GenerateTrajectory();
    nXMax.ValueChanged += (s, e) => GenerateTrajectory();
    nSteps.ValueChanged += (s, e) => GenerateTrajectory();
    numericUpDown1.ValueChanged += (s, e) => GenerateTrajectory();

    nCenterX.ValueChanged += (s, e) => UpdateViewParameters();
}

```

```

nCenterY.ValueChanged += (s, e) => UpdateViewParameters();
nZoom.ValueChanged += (s, e) => UpdateViewParameters();

pictureBox1.MouseDown += PictureBox1_MouseDown;
pictureBox1.MouseMove += PictureBox1_MouseMove;
pictureBox1.MouseUp += PictureBox1_MouseUp;
pictureBox1.MouseWheel += PictureBox1_MouseWheel;
pictureBox1.Click += (s, e) => pictureBox1.Focus();
}

private void GenerateTrajectory()
{
    double xmin = (double)nXMin.Value;
    double xmax = (double)nXMax.Value;
    int rawSteps = (int)nSteps.Value;
    double spacing = (double)numericUpDown1.Value;

    trajectory.Generate(xmin, xmax, rawSteps, spacing);
    pictureBox1.Invalidate();
}

private void UpdateViewParameters()
{
    viewCenter = new PointF((float)nCenterX.Value, (float)nCenterY.Value);
    zoom = (float)nZoom.Value;
    pictureBox1.Invalidate();
}

private void PictureBox1_MouseDown(object sender, MouseEventArgs e)
{
    if (e.Button == MouseButtons.Left)
    {
        mouseDragStart = e.Location;
        viewCenterDragStart = viewCenter;
        isDragging = true;
        pictureBox1.Cursor = Cursors.NoMove2D;
    }
}

private void PictureBox1_MouseMove(object sender, MouseEventArgs e)
{
    if (isDragging && e.Button == MouseButtons.Left)
    {
        float dx = e.Location.X - mouseDragStart.X;
        float dy = e.Location.Y - mouseDragStart.Y;

        float mathDx = dx / zoom;
        float mathDy = dy / zoom;

        float newCenterX = viewCenterDragStart.X - mathDx;
        float newCenterY = viewCenterDragStart.Y + mathDy;

        nCenterX.Value = Math.Max(nCenterX.Minimum, Math.Min(nCenterX.Maximum,
(decimal)newCenterX));
        nCenterY.Value = Math.Max(nCenterY.Minimum, Math.Min(nCenterY.Maximum,
(decimal)newCenterY));
    }
}

private void PictureBox1_MouseUp(object sender, MouseEventArgs e)
{
    if (e.Button == MouseButtons.Left)
    {
        isDragging = false;
        pictureBox1.Cursor = Cursors.Default;
    }
}

```

```

    }
}

private void PictureBox1_MouseWheel(object sender, MouseEventArgs e)
{
    float zoomFactor = 1.15f;
    float currentZoom = (float)nZoom.Value;

    if (e.Delta > 0)
    {
        currentZoom = Math.Min((float)nZoom.Maximum, currentZoom * zoomFactor);
    }
    else
    {
        currentZoom = Math.Max((float)nZoom.Minimum, currentZoom / zoomFactor);
    }

    nZoom.Value = (decimal)currentZoom;
}

private void UpdateBrushComboBoxes()
{
    var selectedCircle = cbCircleBrush.SelectedItem as CustomBrushData;
    var selectedHexagon = cbHexagonBrush.SelectedItem as CustomBrushData;
    var selectedCurve = cbCurveBrush.SelectedItem as CustomBrushData;

    cbCircleBrush.Items.Clear();
    cbHexagonBrush.Items.Clear();
    cbCurveBrush.Items.Clear();

    foreach (var brush in customBrushes)
    {
        cbCircleBrush.Items.Add(brush);
        cbHexagonBrush.Items.Add(brush);
        cbCurveBrush.Items.Add(brush);
    }

    if (selectedCircle != null && cbCircleBrush.Items.Contains(selectedCircle))
        cbCircleBrush.SelectedItem = selectedCircle;
    else if (cbCircleBrush.Items.Count > 0)
        cbCircleBrush.SelectedItem = defaultCircleBrush;

    if (selectedHexagon != null && cbHexagonBrush.Items.Contains(selectedHexagon))
        cbHexagonBrush.SelectedItem = selectedHexagon;
    else if (cbHexagonBrush.Items.Count > 0)
        cbHexagonBrush.SelectedItem = defaultHexagonBrush;

    if (selectedCurve != null && cbCurveBrush.Items.Contains(selectedCurve))
        cbCurveBrush.SelectedItem = selectedCurve;
    else if (cbCurveBrush.Items.Count > 0)
        cbCurveBrush.SelectedItem = defaultCurveBrush;
}

private void RebuildBrushMenu()
{
    while (кистиToolStripMenuItem.DropDownItems.Count > 2)
    {
        кистиToolStripMenuItem.DropDownItems.RemoveAt(2);
    }

    foreach (var brush in customBrushes)

```

```

        {
            var item = new ToolStripMenuItem(brush.ToString());
            item.Tag = brush;
            item.Click += BrushMenuItem_Click;
            кистиToolStripMenuItem.DropDownItems.Add(item);
        }
    }

    private void BrushMenuItem_Click(object sender, EventArgs e)
    {
        if (sender is ToolStripMenuItem item && item.Tag is CustomBrushData brush)
        {
            MessageBox.Show(
                $"Свойства кисти:\n\n" +
                $"Название: {brush.Name}\n" +
                $"Цвет: {brush.Color.Name} (R:{brush.Color.R}, G:{brush.Color.G}, B:
{brush.Color.B})\n" +
                $"Ширина: {brush.Width}px\n" +
                $"Стиль: {brush.DashStyle}",
                "Информация о кисти",
                MessageBoxButtons.OK,
                MessageBoxIcon.Information
            );
        }
    }

    private float GetFloatFromTextBox(TextBox textBox, float defaultValue)
    {
        if (float.TryParse(textBox.Text, out float value))
        {
            return value;
        }
        else
        {
            textBox.Text = defaultValue.ToString();
            return defaultValue;
        }
    }

    #region Колбеки событий

    private void bStartPause_Click(object sender, EventArgs e)
    {
        if (timer1.Enabled)
        {
            timer1.Stop();
            bStartPause.Text = "СТАРТ";
        }
        else
        {
            timer1.Start();
            bStartPause.Text = "ПАУЗА";
        }
    }

    private void bReset_Click(object sender, EventArgs e)
    {
        circleT = 0;
        hexagonT = 200;
        pictureBox1.Invalidate();
    }

    private void создатьКистьToolStripMenuItem_Click(object sender, EventArgs e)
    {
        using (BrushDialog dialog = new BrushDialog())
    
```

```

    {
        if (dialog.ShowDialog() == DialogResult.OK)
        {
            string brushName = $"Кисть #{customBrushes.Count + 1}";
            var newBrush = new CustomBrushData
            {
                Name = brushName,
                Color = dialog.ChosenColor,
                Width = dialog.ChosenWidth,
                DashStyle = dialog.ChosenDashStyle,
                CustomPattern = dialog.ChosenCustomPattern
            };

            customBrushes.Add(newBrush);

            UpdateBrushComboBoxes();
            RebuildBrushMenu();
        }
    }
}

#endregion

private PointF MathToScreen(PointF mathPt)
{
    float x = (pictureBox1.Width / 2f) + (mathPt.X - viewCenter.X) * zoom;
    float y = (pictureBox1.Height / 2f) - (mathPt.Y - viewCenter.Y) * zoom;
    return new PointF(x, y);
}

private PointF ScreenToMath(PointF screenPt)
{
    float x = (screenPt.X - pictureBox1.Width / 2f) / zoom + viewCenter.X;
    float y = (pictureBox1.Height / 2f - screenPt.Y) / zoom + viewCenter.Y;
    return new PointF(x, y);
}

private void timer1_Tick(object sender, EventArgs e)
{
    if (trajectory.EquidistantPoints.Count == 0) return;

    globalTime += 0.025f;

    double speedFactor = GetFloatFromTextBox(tObjectSpeed, 2.5f);

    circleT += speedFactor;
    hexagonT += speedFactor;

    pictureBox1.Invalidate();
}

private void pictureBox1_Paint(object sender, PaintEventArgs e)
{
    Graphics g = e.Graphics;
    g.SmoothingMode = SmoothingMode.AntiAlias;
    g.Clear(Color.FromArgb(255, 255, 255));

    // 1. отрисовка осей и координатной сетки
    DrawAxesAndGrid(g);

    // 2. отрисовка траектории  $x^3 + y^3 = 1$ 
    DrawCurve(g);

    if (trajectory.EquidistantPoints.Count == 0) return;

```

```

        // 3. расчет и отрисовка круг
        DrawCircleObject(g);

        // 4. расчет и отрисовка шестиугольник
        DrawHexagonObject(g);
    }

    private void DrawAxesAndGrid(Graphics g)
    {
        float gridStep = 1.0f;

        using (Pen gridPen = new Pen(Color.FromArgb(50, 50, 50), 1f))
        using (Pen axisPen = new Pen(Color.FromArgb(0, 0, 0), 1.5f))
        using (Font font = new Font("Segoe UI", 8.5f))
        using (Brush textBrush = new SolidBrush(Color.FromArgb(20, 20, 20)))
        {
            PointF minMath = ScreenToMath(new PointF(0, pictureBox1.Height));
            PointF maxMath = ScreenToMath(new PointF(pictureBox1.Width, 0));

            float startX = (float)(Math.Floor(minMath.X / gridStep) * gridStep);
            float endX = (float)(Math.Ceiling(maxMath.X / gridStep) * gridStep);
            float startY = (float)(Math.Floor(minMath.Y / gridStep) * gridStep);
            float endY = (float)(Math.Ceiling(maxMath.Y / gridStep) * gridStep);

            // вертикальная разметка
            for (float x = startX; x <= endX; x += gridStep)
            {
                PointF pStart = MathToScreen(new PointF(x, minMath.Y));
                PointF pEnd = MathToScreen(new PointF(x, maxMath.Y));
                g.DrawLine(gridPen, pStart, pEnd);

                PointF pZeroY = MathToScreen(new PointF(x, 0));
                g.DrawString(x.ToString("0.0"), font, textBrush, pZeroY.X + 2,
Math.Min(pictureBox1.Height - 18, Math.Max(5, pZeroY.Y + 2)));
            }

            // горизонтальная разметка
            for (float y = startY; y <= endY; y += gridStep)
            {
                PointF pStart = MathToScreen(new PointF(minMath.X, y));
                PointF pEnd = MathToScreen(new PointF(maxMath.X, y));
                g.DrawLine(gridPen, pStart, pEnd);

                PointF pZeroX = MathToScreen(new PointF(0, y));
                g.DrawString(y.ToString("0.0"), font, textBrush, Math.Min(pictureBox1.Width -
30, Math.Max(5, pZeroX.X + 5)), pZeroX.Y - 14);
            }

            // ох и оу
            PointF origin = MathToScreen(new PointF(0, 0));
            g.DrawLine(axisPen, 0, origin.Y, pictureBox1.Width, origin.Y);
            g.DrawLine(axisPen, origin.X, 0, origin.X, pictureBox1.Height);
        }
    }

    private void DrawCurve(Graphics g)
    {
        if (trajectory.EquidistantPoints.Count < 2) return;

        CustomBrushData activeBrush = cbCurveBrush.SelectedItem as CustomBrushData ??
defaultCurveBrush;
        using (Pen curvePen = activeBrush.CreatePen())
        {
            List<PointF> screenPoints = new List<PointF>();

```

```

        foreach (var pt in trajectory.EquidistantPoints)
        {
            screenPoints.Add(MathToScreen(pt));
        }
        g.DrawLine(curvePen, screenPoints.ToArray());
    }
}

private void DrawCircleObject(Graphics g)
{
    int count = trajectory.EquidistantPoints.Count;
    int index = ((int)circleT % count + count) % count;
    PointF mathPos = trajectory.EquidistantPoints[index];
    PointF screenPos = MathToScreen(mathPos);

    float rMin = GetFloatFromTextBox(tCircleSmallR, 10f);
    float rMax = GetFloatFromTextBox(tCircleBigR, 40f);
    float pSpeed = GetFloatFromTextBox(tCirclePulseSpeed, 1f);

    if (rMax < rMin) rMax = rMin + 5;

    float currentRadius = rMin + (rMax - rMin) * 0.5f * (1.0f + (float)Math.Sin(globalTime *
pSpeed));

    CustomBrushData activeBrush = cbCircleBrush.SelectedItem as CustomBrushData ??
defaultCircleBrush;

    using (Pen circlePen = activeBrush.CreatePen())
    {
        g.DrawEllipse(circlePen, screenPos.X - currentRadius, screenPos.Y - currentRadius,
currentRadius * 2, currentRadius * 2);
    }

    using (Brush brush = new SolidBrush(activeBrush.Color))
    {
        g.FillEllipse(brush, screenPos.X - 3, screenPos.Y - 3, 6, 6);
    }
}

private void DrawHexagonObject(Graphics g)
{
    int count = trajectory.EquidistantPoints.Count;
    int index = ((int)hexagonT % count + count) % count;
    PointF mathPos = trajectory.EquidistantPoints[index];
    PointF touchPoint = MathToScreen(mathPos);

    float pSpeed = GetFloatFromTextBox(tHexSpeed, 0f);
    float hexR = GetFloatFromTextBox(tHexagonR, 30f);
    float currentRadius = hexR - hexR * (float)Math.Abs(Math.Sin(globalTime * pSpeed));

    float rotSpeed = GetFloatFromTextBox(tHexagonAngleSpeed, 1f);

    float angle = globalTime * rotSpeed;

    PointF[] vertices = new PointF[6];
    for (int i = 0; i < 6; i++)
    {
        float a = angle + i * (float)Math.PI / 3f;
        vertices[i] = new PointF(currentRadius * (float)Math.Cos(a), currentRadius *
(float)Math.Sin(a));
    }

    float dx = touchPoint.X - vertices[0].X;
    float dy = touchPoint.Y - vertices[0].Y;

```

```

        for (int i = 0; i < 6; i++)
        {
            vertices[i].X += dx;
            vertices[i].Y += dy;
        }

        CustomBrushData activeBrush = cbHexagonBrush.SelectedItem as CustomBrushData ??
defaultHexagonBrush;

        using (Pen hexPen = activeBrush.CreatePen())
        {
            g.DrawPolygon(hexPen, vertices);
        }

        using (Brush touchBrush = new SolidBrush(activeBrush.Color))
        {
            g.FillEllipse(touchBrush, touchPoint.X - 4, touchPoint.Y - 4, 8, 8);
        }
    }

    private void panelControls_Paint(object sender, PaintEventArgs e)
    {
    }

    private void формулировкаToolStripMenuItem_Click(object sender, EventArgs e)
    {
        // TODO: Form1 не существует в проекте
        // new Form1().ShowDialog();
    }
}

public class CustomBrushData
{
    public string Name { get; set; }
    public Color Color { get; set; }
    public float Width { get; set; }
    public DashStyle DashStyle { get; set; }
    public float[] CustomPattern { get; set; }

    public override string ToString()
    {
        return $"{Name} ({Color.Name}, {Width}px, {DashStyle})";
    }

    public Pen CreatePen()
    {
        Pen pen = new Pen(Color, Width);
        pen.DashStyle = DashStyle;
        if (DashStyle == DashStyle.Custom && CustomPattern != null)
        {
            pen.DashPattern = CustomPattern;
        }
        return pen;
    }
}

public class TrajectoryPath
{
    public List<PointF> RawPoints { get; private set; } = new List<PointF>();
    public List<PointF> EquidistantPoints { get; private set; } = new List<PointF>();
    public double TotalLength { get; private set; }

    public void Generate(double xMin, double xMax, int rawSteps, double spacing)

```



```

{
    RawPoints.Clear();
    EquidistantPoints.Clear();

    // 1. базовые точки  $y = (1 - x^3)^{1/3}$ 
    for (int i = 0; i <= rawSteps; i++)
    {
        double t = (double)i / rawSteps;
        double x = xMin + t * (xMax - xMin);
        double y = CubeRoot(1.0 - x * x * x);
        RawPoints.Add(new PointF((float)x, (float)y));
    }

    if (RawPoints.Count < 2) return;

    // 2. интегральный расчет длины дуги
    double accumLength = 0;
    List<double> distances = new List<double> { 0 };
    for (int i = 1; i < RawPoints.Count; i++)
    {
        double dx = RawPoints[i].X - RawPoints[i - 1].X;
        double dy = RawPoints[i].Y - RawPoints[i - 1].Y;
        double dist = Math.Sqrt(dx * dx + dy * dy);
        accumLength += dist;
        distances.Add(accumLength);
    }
    TotalLength = accumLength;

    // 3. расстановка точек анимации по равным интервалам
    double currentDist = 0;
    int rawIdx = 0;
    while (currentDist <= TotalLength)
    {
        while (rawIdx < distances.Count - 1 && distances[rawIdx + 1] < currentDist)
        {
            rawIdx++;
        }
        if (rawIdx >= distances.Count - 1)
        {
            EquidistantPoints.Add(RawPoints[RawPoints.Count - 1]);
            break;
        }
        double t = (currentDist - distances[rawIdx]) / (distances[rawIdx + 1] -
distances[rawIdx]);
        float x = (float)(RawPoints[rawIdx].X + t * (RawPoints[rawIdx + 1].X -
RawPoints[rawIdx].X));
        float y = (float)(RawPoints[rawIdx].Y + t * (RawPoints[rawIdx + 1].Y -
RawPoints[rawIdx].Y));
        EquidistantPoints.Add(new PointF(x, y));
        currentDist += spacing;
    }
}

private double CubeRoot(double val)
{
    return val >= 0 ? Math.Pow(val, 1.0 / 3.0) : -Math.Pow(-val, 1.0 / 3.0);
}
}

```

BrushDialog

```

using System;
using System.Collections.Generic;

```

```

using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Drawing.Drawing2D;
using System.Text;
using System.Windows.Forms;

namespace c2_SP_Tasks.Task2
{
    public partial class BrushDialog : Form
    {
        public Color ChosenColor { get; private set; } = Color.Black;
        public float ChosenWidth { get; private set; } = 5;
        public DashStyle ChosenDashStyle { get; private set; } = DashStyle.Solid;
        public float[] ChosenCustomPattern { get; private set; } = null;

        public BrushDialog()
        {
            InitializeComponent();
        }

        private void bColor_Click(object sender, EventArgs e)
        {
            using (ColorDialog cd = new ColorDialog())
            {
                cd.Color = bColor.BackColor;
                if (cd.ShowDialog() == DialogResult.OK)
                {
                    bColor.BackColor = cd.Color;
                    ChosenColor = cd.Color;
                    pictureBox1.Invalidate(); // Заставляем превью перерисоваться
                }
            }
        }

        private void UpdatePreview(object sender, EventArgs e)
        {
            tPattern.Enabled = (cType.SelectedIndex == 4);

            ChosenWidth = (float)nWidth.Value;

            switch (cType.SelectedIndex)
            {
                case 0: ChosenDashStyle = DashStyle.Solid; break;
                case 1: ChosenDashStyle = DashStyle.Dash; break;
                case 2: ChosenDashStyle = DashStyle.Dot; break;
                case 3: ChosenDashStyle = DashStyle.DashDot; break;
                case 4: ChosenDashStyle = DashStyle.Custom; break;
            }

            if (ChosenDashStyle == DashStyle.Custom)
            {
                ChosenCustomPattern = ParsePattern(tPattern.Text);
            }
            else
            {
                ChosenCustomPattern = null;
            }
            pictureBox1.Invalidate();
        }

        private float[] ParsePattern(string text)
        {
            if (string.IsNullOrEmpty(text)) return null;

```

```

try
{
    var numbers = text.Split(new[] { ' ' }, StringSplitOptions.RemoveEmptyEntries)
        .Select(n => float.Parse(n))
        .Where(n => n > 0)
        .ToArray();

    return numbers.Length > 0 ? numbers : null;
}
catch
{
    return null;
}
}

private void PictureBox1_Paint(object sender, PaintEventArgs e)
{
    Graphics g = e.Graphics;
    g.SmoothingMode = SmoothingMode.AntiAlias;

    int diameter = (int)(Math.Min(pictureBox1.Width, pictureBox1.Height) * 0.6);

    int x = (pictureBox1.Width - diameter) / 2;
    int y = (pictureBox1.Height - diameter) / 2;

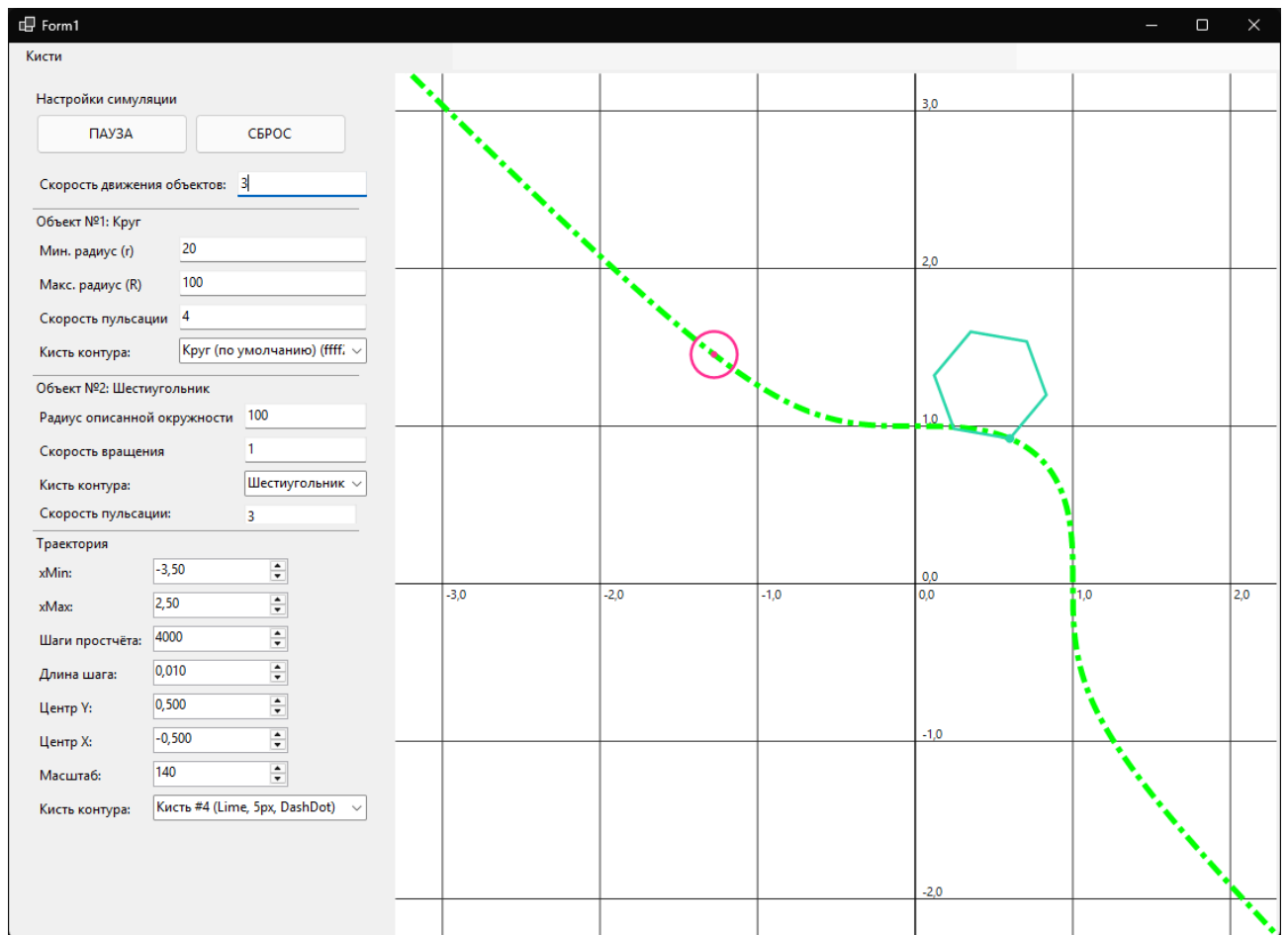
    using (Pen pen = new Pen(ChosenColor, ChosenWidth))
    {
        pen.DashStyle = ChosenDashStyle;

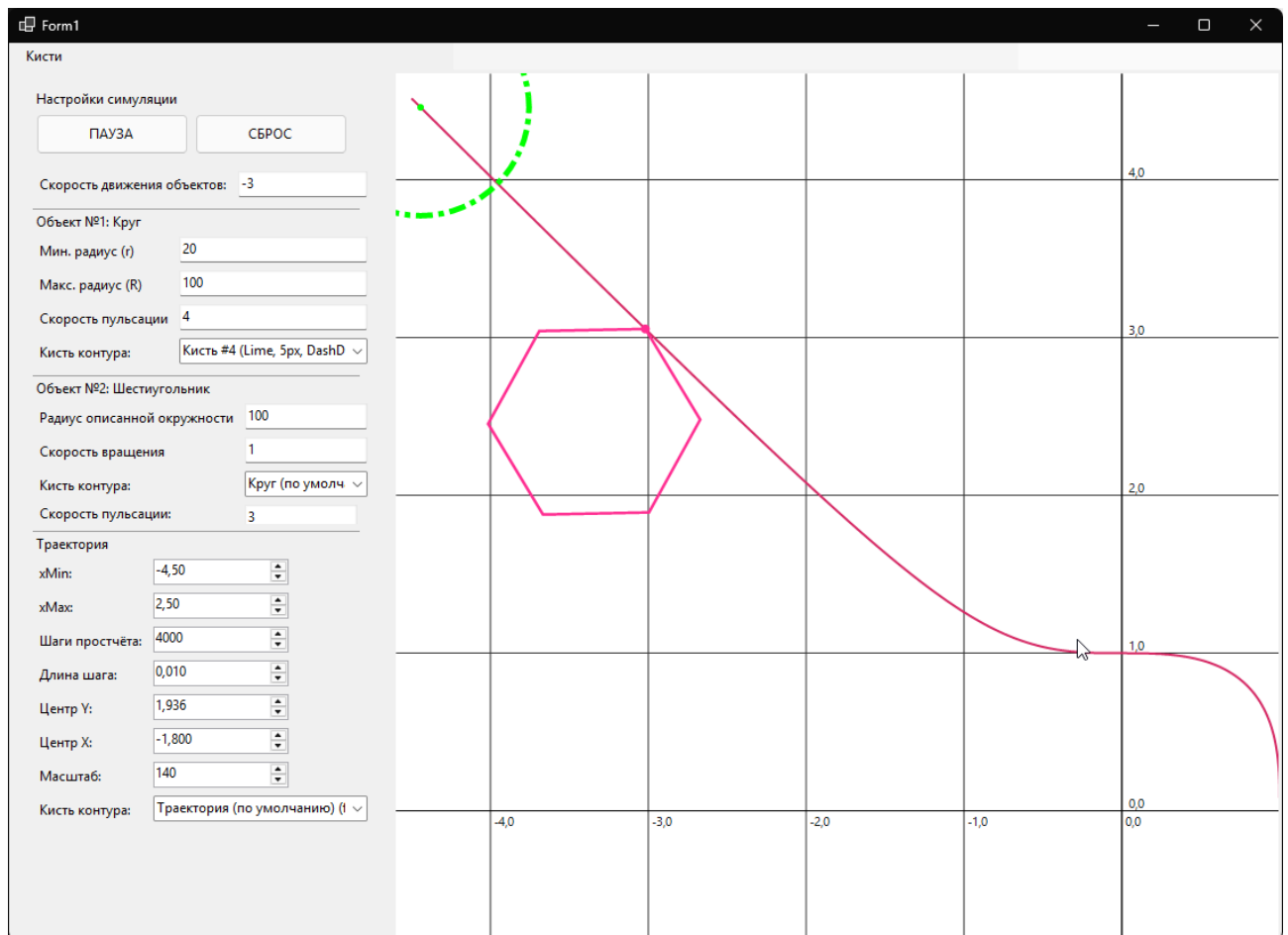
        if (ChosenDashStyle == DashStyle.Custom && ChosenCustomPattern != null)
        {
            pen.DashPattern = ChosenCustomPattern;
        }

        g.DrawEllipse(pen, x, y, diameter, diameter);
    }
}
}
}

```

5. Примеры различного использования





Раздел 3. Второе индивидуальное Задание

1. Формулировка задания

Во втором индивидуальном задании реализована программа для построения фракталов на основе рекурсивной структуры узлов. Программа позволяет:

- строить фрактальную структуру с заданной глубиной рекурсии;
- визуализировать фрактал в главном окне с интерактивным управлением;
- отображать структуру рекурсивного дерева в отдельном окне;
- управлять параметрами построения: масштабом, количеством уровней, размером элементов.

Тип фрактала: снежинка из окружностей (на каждом новом шаге рисуются окружности в углах правильного шестиугольника, описанного около «листовых» элементов).

2. Математическая постановка

В программе Task3 используется рекурсивная математическая модель фрактала, основанная на системе узлов, распределённых в двумерной плоскости. Каждый узел характеризуется координатами, радиусом и уровнем в иерархии.

Структура фрактала

Фрактал представляет собой иерархическую структуру узлов, где корневой узел находится в центре координат:

$$P_0 = (0, 0), \quad R_0 = 100.$$

На уровне 0 (корень) располагаются 6 дочерних узлов, распределённых по 6 направлениям под углами:

$$\theta_i = i \times 60^\circ, \quad i = 0, 1, \dots, 5.$$

На последующих уровнях каждый узел порождает 5 дочерних узлов, которые распределяются относительно входящего направления.

Рекурсивное построение узлов

Для каждого узла-родителя вычисляется радиус дочерних узлов:

$$R_{\text{child}} = R_{\text{parent}} \times s,$$

где s — масштабный коэффициент (scaleFactor). Расстояние от центра родителя до центра потомка:

$$d = R_{\text{parent}} + R_{\text{child}}.$$

На уровне 0 потомки располагаются под углами:

$$\theta_i = i \times 60^\circ.$$

На уровнях > 0 потомки располагаются под углами:

$$\theta_j = (\theta_{\text{in}} + j \times 60^\circ) \bmod 360^\circ, \quad j = -2, -1, 0, 1, 2,$$

где θ_{in} — входящий угол (угол к родителю). Таким образом достигается естественное ветвление структуры.

Координаты дочернего узла:

$$P_{\text{child}} = (x_{\text{parent}} + d \cos(\theta), y_{\text{parent}} + d \sin(\theta)).$$

Расчёт координат в дереве структуры

Для визуализации структуры дерева рекурсии используются специальные координаты TreeX и TreeY. Алгоритм расчёта работает в два прохода:

1. Постоядерный проход (post-order traversal): для листовых узлов назначаются последовательные координаты TreeX, а TreeY определяется глубиной узла:

$$y_{\text{tree}} = \text{depth} \times 100.$$

2. Для внутренних узлов TreeX вычисляется как средняя арифметическая координата дочерних узлов:

$$x_{\text{tree}(\text{parent})} = \frac{1}{n} \sum_{i=1}^n x_{\text{tree}(c_i)},$$

где c_i — дочерние узлы.

Визуализация фрактала

Фрактал рисуется в главном окне с использованием двухмерной трансформации. Для каждого узла вычисляются экранные координаты:

$$x_{\text{screen}} = c_x + x_{\text{fractal}} \times s,$$

$$y_{\text{screen}} = c_y + y_{\text{fractal}} \times s,$$

где (c_x, c_y) — центр экрана, s — масштаб, применяемый при визуализации.

Преобразование координат и масштабирование

Две системы координат используются независимо:

- фрактальные координаты (FractalX, FractalY) — математические координаты в исходной системе;
- координаты дерева структуры (TreeX, TreeY) — условные координаты для визуализации иерархии.

При отображении применяется масштабирование: $s = \left(\frac{\text{scale}}{100}\right)$, где scale вводится пользователем через NumericUpDown. Пользователь может перемещать фрактал левой кнопкой мыши, изменяя смещение offset_x и offset_y .

3. Описание используемых элементов для интерфейса и их настройка
Интерфейс программы Task3 реализован в виде двух окон: основной формы для отображения фрактала (FractalForm) и отдельной формы для визуализации дерева структуры (TreeForm). Основная структура задаётся контейнером TableLayoutPanel и содержит панель управления слева и область рисования справа.

Основная область рисования (PictureBox1)

Это главный элемент визуализации. Элемент поддерживает:

- отрисовку узлов фрактала в виде эллипсов с заливкой и контуром;
- цветовую кодировку по уровню дерева из массива LevelColors;

- интерактивное управление через мышь: перемещение при левой кнопке и масштабирование при колесе.

Панель управления параметрами

Левая панель содержит следующие элементы управления:

- NumericUpDown1 — максимальный уровень рекурсии (от 0 до количества уровней);
- NumericUpDown2 — масштаб отображения (в процентах);
- NumericUpDown3 — масштабный коэффициент для уменьшения размера узлов на каждом уровне;
- Button1 — кнопка «Построить» для инициирования построения и отрисовки фрактала.
- Button2 — кнопка вызова постановки задания.

При нажатии кнопки вызывается метод BuildAndRender(), который создаёт корневой узел, рекурсивно строит структуру и вызывает методы расчёта и отрисовки.

Форма отображения дерева структуры (TreeForm)

При построении фрактала автоматически открывается дополнительное окно, в котором визуализируется иерархическая структура рекурсивного дерева.

Форма содержит:

- PictureBox для отображения дерева;
- методы расчёта границ дерева и масштабирования для полного отображения;
- горизонтальные пунктирные линии, указывающие на уровни дерева с их номерами;
- рёбра (связи) между узлами, окрашенные в цвета уровней;
- узлы, отображаемые в виде окрашенных эллипсов с белой границей.

Взаимодействие с мышью

FractalForm поддерживает интерактивное управление:

- левая кнопка мыши + перемещение: сдвиг области просмотра (изменение _offsetX и _offsetY);
- курсор: при нажатии левой кнопки изменяется на NoMove2D для наглядности;
- при отпускании кнопки курсор возвращается в нормальное состояние.

TreeForm срабатывает событие ResizeEnd для перерисовки дерева при изменении размера окна.

4. Описание используемых примитивов

В программе используются следующие графические примитивы:

- эллипс с заливкой и контуром: для отображения узлов фрактала используются методы FillEllipse (с полупрозрачной заливкой) и DrawEllipse (для контура). Цвет зависит от уровня узла в иерархии;
- линия: метод DrawLine применяется для визуализации связей между узлами в окне дерева (рёбра в графе);
- горизонтальные пунктирные линии: служат для обозначения уровней в дереве структуры и облегчения чтения иерархии;
- текст: метод DrawString используется для вывода номеров уровней на левой стороне TreeForm;
- фоновый прямоугольник под текстом: FillRectangle применяется для улучшения читаемости текста на фоне линий.

5. Код программы

FractalForm

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Drawing.Drawing2D;
using System.Text;
using System.Windows.Forms;

namespace c2_SP_Tasks.Task3
{
    public partial class FractalForm : Form
    {
        FractalNode _rootNode;
        TreeForm _treeForm;
        int _xStepCounter = 0;

        float _offsetX, _offsetY;

        public static readonly Color[] LevelColors = new Color[]
        {
            Color.FromArgb(231, 76, 60),
            Color.FromArgb(230, 126, 34),
            Color.FromArgb(46, 204, 113),
            Color.FromArgb(52, 152, 219),
            Color.FromArgb(155, 89, 182),
            Color.FromArgb(26, 188, 156),
            Color.FromArgb(241, 196, 15)
        };

        public FractalForm()
        {
            InitializeComponent();
        }

        private void button1_Click(object sender, EventArgs e)
        {
            BuildAndRender();
        }

        private void pictureBox1_Resize(object sender, EventArgs e)
        {

```

```

{
    DrawFractal();
}

private void FractalForm_FormClosed(object sender, FormClosedEventArgs e)
{
    if (_treeForm != null && !_treeForm.IsDisposed)
    {
        _treeForm.Close();
    }
}

private void BuildAndRender()
{
    try
    {
        int maxLevel = (int)numericUpDown1.Value;

        // Инициализация корневого элемента фрактала (центральный круг)
        _rootNode = new FractalNode
        {
            FractalX = 0,
            FractalY = 0,
            Radius = 100f,
            Level = 0
        };

        double scaleFactor = (double)numericUpDown3.Value;
        GenerateFractalTree(_rootNode, maxLevel, -1, scaleFactor);

        _xStepCounter = 0;
        CalculateInitialTreeCoords(_rootNode, 0);

        DrawFractal();

        if (_treeForm == null || _treeForm.IsDisposed)
        {
            _treeForm = new TreeForm();
        }

        _treeForm.Show();
        _treeForm.UpdateTree(_rootNode);
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Ошибка построения фрактала: {ex.Message}", "Ошибка",
            MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}

private void GenerateFractalTree(FractalNode parent, int maxLevel, double incomingAngle,
double scale)
{
    if (parent.Level >= maxLevel) return;

    float childRadius = parent.Radius * (float)scale;
    float distance = parent.Radius + childRadius;

    if (parent.Level == 0)
    {
        for (int i = 0; i < 6; i++)
        {
            double angleDeg = i * 60.0;
            double angleRad = angleDeg * Math.PI / 180.0;

```

```

        var child = new FractalNode
        {
            FractalX = parent.FractalX + distance * (float)Math.Cos(angleRad),
            FractalY = parent.FractalY + distance * (float)Math.Sin(angleRad),
            Radius = childRadius,
            Level = parent.Level + 1
        };
        parent.Children.Add(child);
        GenerateFractalTree(child, maxLevel, angleDeg, scale);
    }
}
else
{
    for (int i = -2; i <= 2; i++)
    {
        double angleDeg = (incomingAngle + i * 60.0) % 360.0;
        if (angleDeg < 0) angleDeg += 360.0;
        double angleRad = angleDeg * Math.PI / 180.0;

        var child = new FractalNode
        {
            FractalX = parent.FractalX + distance * (float)Math.Cos(angleRad),
            FractalY = parent.FractalY + distance * (float)Math.Sin(angleRad),
            Radius = childRadius,
            Level = parent.Level + 1
        };
        parent.Children.Add(child);
        GenerateFractalTree(child, maxLevel, angleDeg, scale);
    }
}
}

private void CalculateInitialTreeCoords(FractalNode node, int depth)
{
    if (node == null) return;

    node.TreeY = depth * 100;

    if (node.Children.Count == 0)
    {
        node.TreeX = _xStepCounter * 50;
        _xStepCounter++;
    }
    else
    {
        foreach (var child in node.Children)
        {
            CalculateInitialTreeCoords(child, depth + 1);
        }

        float sumX = 0;
        foreach (var child in node.Children)
        {
            sumX += child.TreeX;
        }
        node.TreeX = sumX / node.Children.Count;
    }
}

private void DrawFractal()
{
    if (_rootNode == null) return;

    int width = pictureBox1.Width;
    int height = pictureBox1.Height;

```

```

        if (width <= 0 || height <= 0) return;

        // Использование фонового Bitmap для исключения мерцания
        Bitmap bmp = new Bitmap(width, height);
        using (Graphics g = Graphics.FromImage(bmp))
        {
            g.SmoothingMode = SmoothingMode.AntiAlias;
            g.Clear(Color.FromArgb(255, 255, 255));

            float scaleFactor = (float)numericUpDown2.Value / 100f;

            float centerX = width / 2f + _offsetX;
            float centerY = height / 2f + _offsetY;

            RenderFractalNode(g, _rootNode, centerX, centerY, scaleFactor);
        }

        pictureBox1.Image?.Dispose();
        pictureBox1.Image = bmp;
    }

    private void RenderFractalNode(Graphics g, FractalNode node, float cx, float cy, float
scale)
    {
        if (node == null) return;

        float screenX = cx + node.FractalX * scale;
        float screenY = cy + node.FractalY * scale;
        float screenRadius = node.Radius * scale;

        Color color = LevelColors[node.Level % LevelColors.Length];

        using (Pen pen = new Pen(color, 1.5f))
        using (Brush brush = new SolidBrush(Color.FromArgb(40, color)))
        {
            float diameter = screenRadius * 2;
            g.FillEllipse(brush, screenX - screenRadius, screenY - screenRadius, diameter,
diameter);
            g.DrawEllipse(pen, screenX - screenRadius, screenY - screenRadius, diameter,
diameter);
        }

        foreach (var child in parentChildNode(node))
        {
            RenderFractalNode(g, child, cx, cy, scale);
        }
    }

    private List<FractalNode> parentChildNode(FractalNode node)
    {
        return node.Children;
    }

    bool _isDragging = false;
    Point _lastMousePos;

    private void pictureBox1_MouseDown(object sender, MouseEventArgs e)
    {
        if (e.Button == MouseButtons.Left)
        {
            _isDragging = true;
            _lastMousePos = e.Location;
            pictureBox1.Cursor = Cursors.NoMove2D; // Интуитивно понятный курсор переноса
        }
    }

```

```

    }

    private void pictureBox1_MouseUp(object sender, MouseEventArgs e)
    {
        if (e.Button == MouseButtons.Left)
        {
            _isDragging = false;
            pictureBox1.Cursor = Cursors.Default;
        }
    }

    private void pictureBox1_MouseMove(object sender, MouseEventArgs e)
    {
        if (_isDragging && _rootNode != null)
        {
            // Вычисляем дельту сдвига мыши
            float deltaX = e.X - _lastMousePos.X;
            float deltaY = e.Y - _lastMousePos.Y;

            _offsetX += deltaX;
            _offsetY += deltaY;

            _lastMousePos = e.Location;
            DrawFractal();
        }
    }

    private void button2_Click(object sender, EventArgs e)
    {
        new TaskDialog().ShowDialog();
    }
}

public class FractalNode
{
    public float FractalX { get; set; }
    public float FractalY { get; set; }
    public float Radius { get; set; }
    public int Level { get; set; }

    public float TreeX { get; set; }
    public float TreeY { get; set; }

    public List<FractalNode> Children { get; set; } = new List<FractalNode>();
}

```

TreeForm

```

using System;
using System.Drawing;
using System.Drawing.Drawing2D;
using System.Windows.Forms;

namespace c2_SP_Tasks.Task3
{
    public partial class TreeForm : Form
    {
        private FractalNode _root;

        public TreeForm()

```

```

{
    InitializeComponent();
}

public void UpdateTree(FractalNode newRoot)
{
    _root = newRoot;
    DrawTree();
}

private void DrawTree()
{
    if (_root == null) return;

    int width = pictureBox1.Width;
    int height = pictureBox1.Height;

    if (width <= 0 || height <= 0) return;

    Bitmap bmp = new Bitmap(width, height);
    using (Graphics g = Graphics.FromImage(bmp))
    {
        g.SmoothingMode = SmoothingMode.AntiAlias;
        g.Clear(Color.FromArgb(255, 255, 255));

        float minX = float.MaxValue, maxX = float.MinValue;
        float minY = float.MaxValue, maxY = float.MinValue;
        FindTreeBounds(_root, ref minX, ref maxX, ref minY, ref maxY);

        float margin = 40f;
        float diffX = maxX - minX;
        float diffY = maxY - minY;

        float scaleX = diffX == 0 ? 1f : (width - 2 * margin) / diffX;
        float scaleY = diffY == 0 ? 1f : (height - 2 * margin) / diffY;

        RenderLevelScale(g, minY, scaleY, margin, width, GetMaxLevel(_root));

        RenderTreeEdges(g, _root, minX, minY, scaleX, scaleY, margin);

        RenderTreeNode(g, _root, minX, minY, scaleX, scaleY, margin);
    }

    pictureBox1.Image?.Dispose();
    pictureBox1.Image = bmp;
}

private int GetMaxLevel(FractalNode node)
{
    if (node == null) return 0;
    int max = node.Level;
    foreach (var child in node.Children)
    {
        max = Math.Max(max, GetMaxLevel(child));
    }
    return max;
}

private void RenderLevelScale(Graphics g, float minY, float scaleY, float width, float
canvasWidth, int maxLevel)
{
    for (int lvl = 0; lvl <= maxLevel; lvl++)
    {
        float rawY = lvl * 100;
        float screenY = width + (rawY - minY) * scaleY;
    }
}

```

```

        Color levelColor = FractalForm.LevelColors[lvl % FractalForm.LevelColors.Length];

        using (Pen scalePen = new Pen(Color.FromArgb(45, levelColor), 1.2f))
        {
            scalePen.DashStyle = DashStyle.Dash;
            g.DrawLine(scalePen, 15f, screenY, canvasWidth - 15f, screenY);
        }

        string labelText = $"{lvl}";
        using (Font font = new Font("Segoe UI", 8.5f, FontStyle.Bold))
        using (Brush textBrush = new SolidBrush(Color.FromArgb(180, levelColor)))
        using (Brush bgBrush = new SolidBrush(Color.FromArgb(180, Color.White)))
        {
            SizeF textSize = g.MeasureString(labelText, font);

            g.FillRectangle(bgBrush, 20f, screenY - textSize.Height / 2f, textSize.Width +
1f, textSize.Height);

            g.DrawString(labelText, font, textBrush, 23f, screenY - textSize.Height / 2f +
1f);
        }
    }
}

private void FindTreeBounds(FractalNode node, ref float minX, ref float maxX, ref float
minY, ref float maxY)
{
    if (node == null) return;

    if (node.TreeX < minX) minX = node.TreeX;
    if (node.TreeX > maxX) maxX = node.TreeX;
    if (node.TreeY < minY) minY = node.TreeY;
    if (node.TreeY > maxY) maxY = node.TreeY;

    foreach (var child in node.Children)
    {
        FindTreeBounds(child, ref minX, ref maxX, ref minY, ref maxY);
    }
}

private void RenderTreeEdges(Graphics g, FractalNode node, float minX, float minY, float
scaleX, float scaleY, float margin)
{
    if (node == null) return;

    float parentX = margin + (node.TreeX - minX) * scaleX;
    float parentY = margin + (node.TreeY - minY) * scaleY;

    foreach (var child in node.Children)
    {
        float childX = margin + (child.TreeX - minX) * scaleX;
        float childY = margin + (child.TreeY - minY) * scaleY;

        Color edgeColor = FractalForm.LevelColors[child.Level %
FractalForm.LevelColors.Length];

        using (Pen pen = new Pen(Color.FromArgb(160, edgeColor), 1.8f))
        {
            g.DrawLine(pen, parentX, parentY, childX, childY);
        }

        RenderTreeEdges(g, child, minX, minY, scaleX, scaleY, margin);
    }
}

```

```

private void RenderTreeNodes(Graphics g, FractalNode node, float minX, float minY, float
scaleX, float scaleY, float margin)
{
    if (node == null) return;

    float screenX = margin + (node.TreeX - minX) * scaleX;
    float screenY = margin + (node.TreeY - minY) * scaleY;

    Color nodeColor = FractalForm.LevelColors[node.Level % FractalForm.LevelColors.Length];
    float nodeSize = 10f;

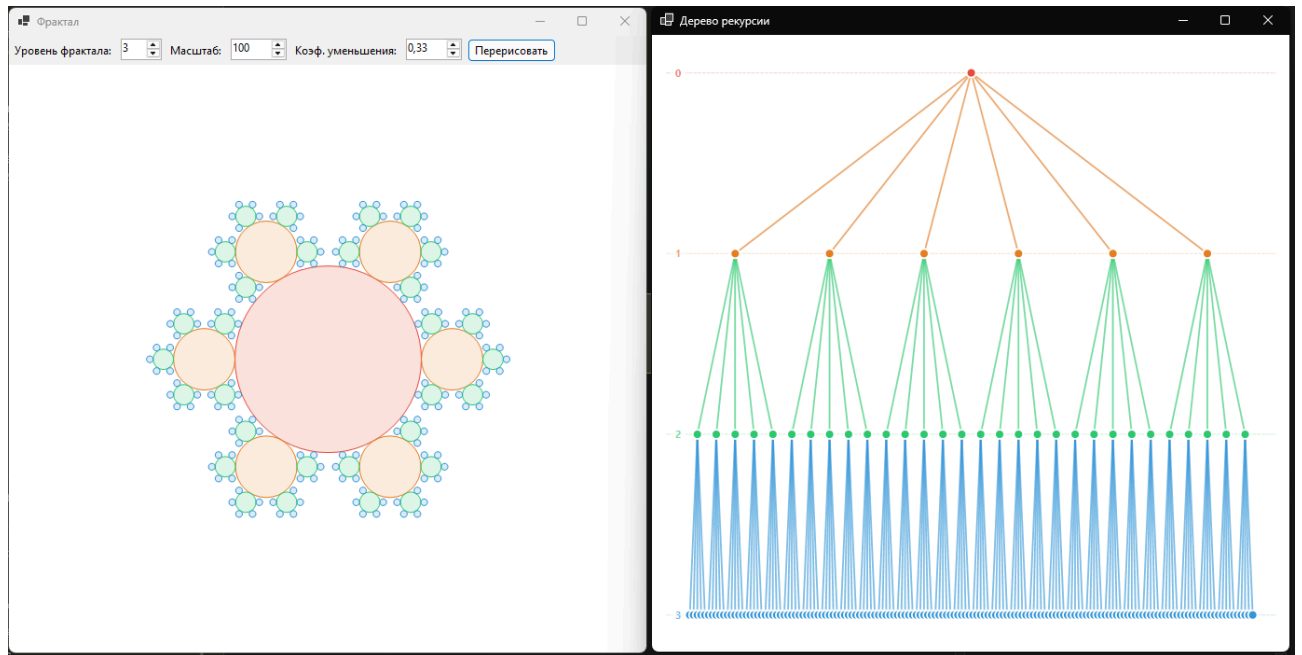
    using (Brush brush = new SolidBrush(nodeColor))
    using (Pen borderPen = new Pen(Color.White, 1.8f))
    {
        g.FillEllipse(brush, screenX - nodeSize / 2f, screenY - nodeSize / 2f, nodeSize,
nodeSize);
        g.DrawEllipse(borderPen, screenX - nodeSize / 2f, screenY - nodeSize / 2f, nodeSize,
nodeSize);
    }

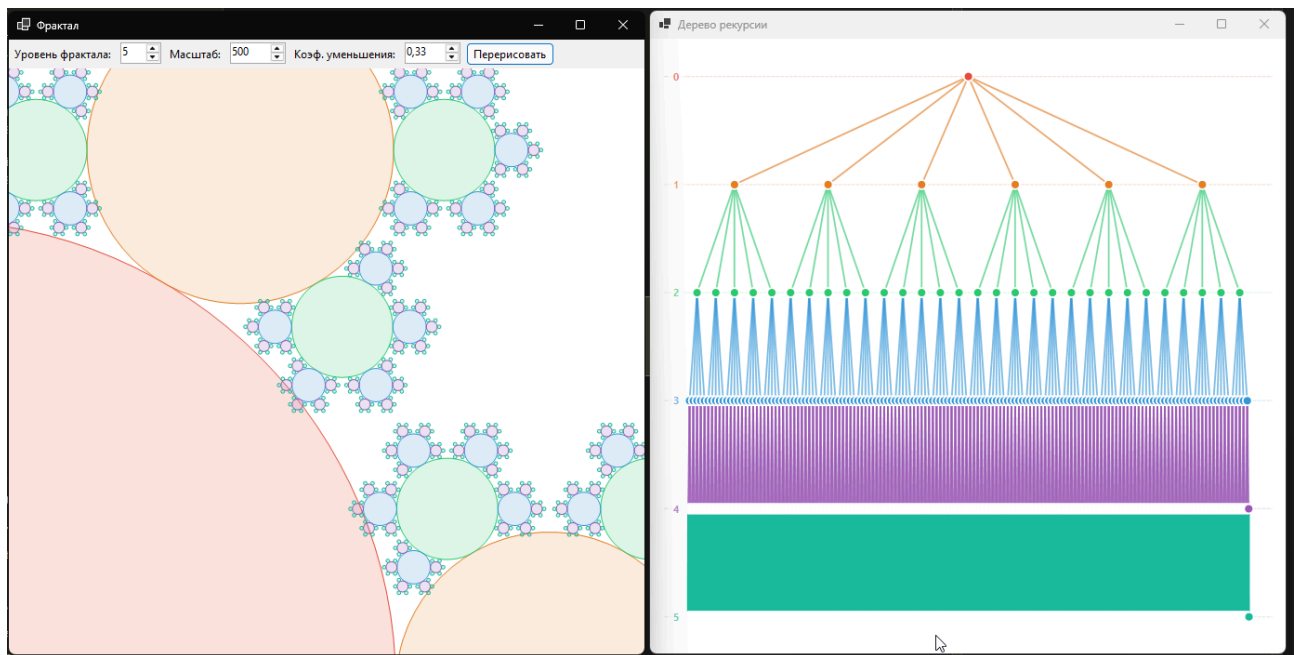
    foreach (var child in node.Children)
    {
        RenderTreeNodes(g, child, minX, minY, scaleX, scaleY, margin);
    }
}

private void TreeForm_ResizeEnd(object sender, EventArgs e)
{
    pictureBox1.Invalidate();
    DrawTree();
}
}

```

6. Примеры различного использования





Раздел 4. Заключение

В ходе прохождения учебной практики были успешно реализованы все три поставленные задания, направленные на освоение средств визуализации и объектно-ориентированного программирования в среде C#.

Первое задание позволило получить практический опыт работы с графическими компонентами Windows Forms, включая управление элементами пользовательского интерфейса, обработку событий, работу с изображениями и базовыми графическими примитивами. Эти навыки составили основу для последующих, более сложных работ.

Второе задание продемонстрировало применение математических моделей в программировании. Реализованная программа анимации показала, как теория (математические уравнения кривых, синусоидальные функции) может быть эффективно преобразована в практический код с использованием геометрических преобразований и интерактивного управления. Особое внимание было уделено равномерному распределению точек по длине дуги, что обеспечило плавное и реалистичное движение объектов.

Третье задание представило собой наиболее сложную часть практики, требующую понимания рекурсивных структур данных и их визуализации. Разработанная программа построения фракталов демонстрирует мощь рекурсии при создании сложных, самоподобных структур. Реализация двойной системы координат и двух окон представления показала, как одну математическую модель можно представить несколькими различными способами.

Все три приложения реализованы с применением принципов объектно-ориентированного программирования, обеспечивают удобный пользовательский интерфейс, поддерживают интерактивное управление и демонстрируют эффективное использование графических библиотек платформы .NET.

В результате проделанной работы были достигнуты основные цели практики:

- освоены методы разработки графических приложений на C#;
- получены навыки преобразования математических моделей в компьютерный код;
- приобретен опыт работы с рекурсивными структурами и их визуализацией;
- развито понимание взаимодействия математики и программирования при создании визуальных систем;
- закрепились знания в области объектно-ориентированного проектирования.

Правильное сочетание теоретических знаний (математика, алгоритмы) с практическими навыками программирования позволяет создавать высокофункциональные и визуально привлекательные приложения.

Полученный опыт может быть применен при разработке более сложных графических систем, игровых движков, систем моделирования и других приложений, требующих работы с визуализацией и компьютерной графикой.